

Sprint Low-Latency - 23 aout -> 20 septembre

Programme jour par jour pour livrer le BIG project sans abandonner les screenings

OBJECTIF

Construire une preuve d'ingenierie end-to-end: performance, correctness, failure handling, mesure et explicabilite.

- 60% projet / 20% algorithmique-OA / 10% C++-Linux-perf / 10% candidatures-visibilite.
- Objectif release v1.0: dimanche 20 septembre.
- Publication finale recommandee: lundi 21 ou mardi 22 septembre.

September 2026 build plan

Regles du sprint

Regle	Application
1. Le chemin critique gagne	Finir end-to-end avant d'ajouter du luxe.
2. Mesure avant optimisation	Aucune "optimisation" sans baseline + mesure.
3. Correctness avant bragging	Tests/invariants/replay avant chiffres LinkedIn.
4. LeetCode reste	15-20% hebdomadaire; pas de grind encyclopedique.
5. Publication progressive	Montrer des milestones; ne pas disparaître jusqu'au 20.
6. Scope kill-switch	Si retard >2 jours, couper SHOULD puis v0.2; jamais sacrifier le MUST.

LEETCODE / ALGO Oui: tu continues. Le quota de 20% du screenshot est raisonnable comme plafond. En pratique: 3 mediums chronometres par semaine + 1 simulation OA courte, avec review des erreurs. Le projet reste dominant.

Template de journee

Si tu as...	Projet 60%	Algo/OA 20%	C++/Linux 10%	Jobs/LinkedIn 10%
6 h nettes	3 h 35	1 h 10	35 min	35 min
8 h nettes	4 h 50	1 h 35	50 min	50 min

- Ce sont des moyennes hebdomadaires, pas une obligation de chronometrer chaque jour a la minute.
- Les jours d'integration peuvent etre 80% projet; les jours OA peuvent remonter l'algo.
- Chaque soir: commit propre + 3 lignes dans BUILDLOG.md: fait / mesure / prochain blocage.

Cadence algorithmique

Cadence	Contenu
Lun / Mer / Ven	1 medium chronometre (35-45 min) + 15 min de post-mortem.
Samedi	Simulation OA 60-75 min: 2 problemes ou 1 dense + correction.
Mardi / Jeudi	Recall léger: 20-30 min si besoin; sinon reporter au projet.
Dimanche	Pas de nouveau pattern; seulement correction d'une erreur recurrente si energie.

- Priorites: arrays/hash maps, two pointers/sliding window, binary search, heaps, intervals, graphs/BFS-Dijkstra-topo, DP classique, parsing/state.
- Eviter de passer 2h sur une nouveaute exotique pendant le sprint. Le but est la vitesse d'execution sur ce que tu sais deja.

Roadmap macro

Bloc	Dates	Milestone
A. Foundations + market data	23-30 aout	Exchange sim, protocoles, UDP feed, sequencing/gap recovery, premier E2E market-data.
B. Hot path + book	31 aout-6 sept.	SPSC, atomics, pinning, order book baseline/optimized, memory experiments.
C. Execution path	7-13 sept.	Strategy, risk, OMS, TCP gateway, fills/rejects, full trading round-trip.
D. Resilience + Reporter + release	14-20 sept.	Journal/replay, crash recovery, backpressure, perf Reporter, tests, docs, v1.0.

Bloc A - Foundations + market data

But: avoir un flux de marche reel, deterministe et recuperable avant toute optimisation.

Dimanche 23 aout	Architecture freeze + repo skeleton
BUILD 60%	CMake presets, targets engine/exchange/bench/tests; interfaces; scenario/config skeleton; BUILDLOG.
ALGO/OA 20%	Pas de LeetCode nouveau; 30 min recall structures + binary search.
C++/LINUX 10%	Compiler flags Release/Debug, warnings, sanitizers strategy.
VISIBILITE 10%	Nettoyer GitHub; README 10 lignes: problem, architecture, target release.
DONE	Repo buildable; empty components link; one command runs a smoke test.

Lundi 24 aout	Binary protocol spec v0
BUILD 60%	Header, message types, endian helpers, encode/decode; golden byte vectors.
ALGO/OA 20%	Medium timed: parsing/hash-map ou array state.
C++/LINUX 10%	Memory layout, padding, alignment, sizeof/static_assert.
VISIBILITE 10%	Identifier 5-8 roles cibles; noter mots-cles qui reviennent.
DONE	Protocol tests green; malformed length/type rejected safely.

Mardi 25 aout	Exchange simulator core
BUILD 60%	Deterministic event generator + minimal matching/ACK/fill model; seed control.
ALGO/OA 20%	Review medium + 1 easy speed drill si necessaire.
C++/LINUX 10%	RAII sockets/resources; ownership boundaries.
VISIBILITE 10%	Ecrire brouillon du post #1 (sans publier encore).
DONE	Same seed -> same event stream and fills.

Bloc A - Foundations + market data - suite

Mercredi 26 aout	UDP multicast publisher
BUILD 60%	Exchange publishes binary market data over UDP; client smoke receiver.
ALGO/OA 20%	Medium timed: sliding window/two pointers.
C++/LINUX 10%	UDP/IP, multicast membership, SO_RCVBUF, non-blocking sockets.
VISIBILITE 10%	1 candidature ciblee + 10 min engagement LinkedIn technique.
DONE	Separate processes exchange/client exchange packets reliably on localhost/LAN.

Jeudi 27 aout	Feed handler RX + decoder
BUILD 60%	Receive loop, validate, decode, normalize events; counters.
ALGO/OA 20%	30 min graph recall ou correction d'une erreur recurrente.
C++/LINUX 10%	epoll basics, recv path, timestamps, syscall cost.
VISIBILITE 10%	Mettre a jour README architecture avec feed path.
DONE	Known packet stream produces exact normalized event sequence.

Vendredi 28 aout	Sequencing + fault injection v0
BUILD 60%	expected sequence, duplicate/out-of-order/gap detection; drop/duplicate/reorder fault modes.
ALGO/OA 20%	Medium timed: graph/BFS/Dijkstra selon rotation.
C++/LINUX 10%	Error handling sans exceptions dans hot loop si choisi; branch behavior.
VISIBILITE 10%	1 candidature ciblee.
DONE	Tests cover normal, duplicate, reorder and one missing sequence.

Bloc A - Foundations + market data - suite

Samedi 29 aout	Market-data recovery
BUILD 60%	Book/feed stale state; snapshot/replay mechanism; resume incrementals.
ALGO/OA 20%	Simulation OA 60-75 min.
C++/LINUX 10%	State machines + idempotency principles.
VISIBILITE 10%	Capturer screenshot/diagram du recovery pour futur post.
DONE	Injected gap recovers and final state matches no-fault run.

Dimanche 30 aout	Integration checkpoint #1
BUILD 60%	Run exchange -> UDP -> feed -> recovery; baseline throughput/latency harness.
ALGO/OA 20%	Correction OA uniquement.
C++/LINUX 10%	Benchmark hygiene: warmup, Release build, pinning caveats, clock choice.
VISIBILITE 10%	Polir post #1 + repo landing page.
DONE	Milestone A demoable; first benchmark JSON stored with commit/config.

Bloc B - Hot path + order book

But: construire le coeur concurrent/cache-aware et commencer les experiences reproductibles.

Lundi 31 aout	SPSC ring buffer baseline
BUILD 60%	Bounded SPSC queue + unit/stress tests; clear overflow policy.
ALGO/OA 20%	Medium timed: heap/intervals.
C++/LINUX 10%	Atomics overview; producer/consumer ownership invariant.
VISIBILITE 10%	PUBLIER POST #1: architecture + objectif + first working market-data path.
DONE	Queue passes wraparound/full/empty/stress tests.

Mardi 1 septembre	Memory ordering correctness
BUILD 60%	Acquire/release implementation and documentation; compare safe baseline.
ALGO/OA 20%	30 min DP recall ou correction.
C++/LINUX 10%	C++ memory model: happens-before, acquire/release, false sharing.
VISIBILITE 10%	1 candidature + repondre aux commentaires utiles du post.
DONE	Comment/invariant explains why SPSC is correct; tests stable.

Mercredi 2 septembre	CPU affinity + thread topology
BUILD 60%	Pin RX and consumer threads; collect pinned/unpinned baseline.
ALGO/OA 20%	Medium timed: binary search / monotonic predicate.
C++/LINUX 10%	sched_setaffinity, context switches, CPU isolation concepts.
VISIBILITE 10%	Update repo with experiment note #1.
DONE	Reporter stub records CPU mapping + latency/throughput deltas.

Bloc B - Hot path + order book - suite

Jeudi 3 septembre	Order book baseline
BUILD 60%	Correct L2/L3 model chosen; add/modify/cancel/trade; invariants.
ALGO/OA 20%	30 min hash-map/string drill.
C++/LINUX 10%	Data locality vs pointer chasing; complexity vs constants.
VISIBILITE 10%	1 candidature ciblee.
DONE	Deterministic book tests + best bid/ask invariants.

Vendredi 4 septembre	Cache-aware order book
BUILD 60%	Second representation or optimized storage; same interface/workload.
ALGO/OA 20%	Medium timed: graph/topological sort.
C++/LINUX 10%	Cache lines, LLC misses, branch prediction; perf stat plan.
VISIBILITE 10%	Draft Post #2 results structure.
DONE	A/B benchmark exists; no conclusion unless repeatable.

Samedi 5 septembre	Memory pool + allocation instrumentation
BUILD 60%	Preallocate hot-path objects; count allocations; remove obvious dynamic churn.
ALGO/OA 20%	Simulation OA 60-75 min.
C++/LINUX 10%	Arena/object pool/free-list tradeoffs; page faults basics.
VISIBILITE 10%	1 candidature ou recruter outreach si pertinent.
DONE	Hot-path allocation count reported; correctness unchanged.

Bloc B - Hot path + order book - suite

Dimanche 6 septembre	Integration checkpoint #2
BUILD 60%	UDP -> feed -> SPSC -> book; run tree/flat, pinned/unpinned, allocation experiments.
ALGO/OA 20%	Correction OA uniquement.
C++/LINUX 10%	perf stat: cycles/instructions/cache/branches/context switches.
VISIBILITE 10%	Finaliser Post #2 + benchmark table.
DONE	Milestone B demoable; 3 reproducible experiment results stored.

Bloc C - Execution path

But: fermer la boucle trading complete jusqu'aux ACK/FILL et tester les transitions OMS.

Lundi 7 septembre	Strategy stub + pre-trade risk
BUILD 60%	Simple deterministic signal; risk rules and kill switch; risk latency delta.
ALGO/OA 20%	Medium timed: sliding window / state.
C++/LINUX 10%	Branching and fast-path design; config outside hot path.
VISIBILITE 10%	PUBLIER POST #2: gap recovery + SPSC/book performance result.
DONE	Signal produces valid intents; risk rejects known bad orders.

Mardi 8 septembre	OMS state machine
BUILD 60%	Order IDs, states, quantities; transition table; impossible-event handling.
ALGO/OA 20%	30 min DP/classic state machine recall.
C++/LINUX 10%	State-machine design, enum layout, invariants.
VISIBILITE 10%	1 candidature ciblee.
DONE	OMS unit tests cover ACK/reject/partial/fill/cancel/timeout.

Mercredi 9 septembre	TCP execution gateway client
BUILD 60%	Non-blocking connect/read/write, framing, partial IO, heartbeat.
ALGO/OA 20%	Medium timed: intervals/heap.
C++/LINUX 10%	TCP handshake, retransmission, Nagle/TCP_NODELAY, send/recv buffers.
VISIBILITE 10%	README: explain UDP feed vs TCP order entry.
DONE	Client survives fragmented frames and partial writes.

Bloc C - Execution path - suite

Jeudi 10 septembre	Exchange TCP order-entry server
BUILD 60%	Accept orders, ACK/reject/fill/cancel; deterministic delays/faults.
ALGO/OA 20%	30 min graph recall.
C++/LINUX 10%	epoll state machines for multiple FDs; disconnect semantics.
VISIBILITE 10%	1 candidature + 10 min LinkedIn engagement.
DONE	Client/server protocol passes integration tests.

Vendredi 11 septembre	Full trading round-trip
BUILD 60%	Book -> strategy -> risk -> OMS -> TCP -> exchange -> ACK/FILL -> OMS.
ALGO/OA 20%	Medium timed: BFS/Dijkstra or parsing.
C++/LINUX 10%	Timestamp placement; end-to-end latency definition.
VISIBILITE 10%	Capture architecture GIF/screenshots/log snippets for final content.
DONE	One scenario completes full round-trip with coherent final position/order state.

Samedi 12 septembre	Reconnect + pathological execution events
BUILD 60%	TCP disconnect/reconnect, heartbeat timeout, duplicate ACK, unknown fill, delayed reject.
ALGO/OA 20%	Simulation OA 60-75 min.
C++/LINUX 10%	Idempotency/reconciliation; retry semantics.
VISIBILITE 10%	1 candidature ciblée.
DONE	No corrupted OMS state under injected execution faults.

Bloc C - Execution path - suite

Dimanche 13 septembre	Integration checkpoint #3
BUILD 60%	Stress full path; fix correctness bugs; stage latency markers added.
ALGO/OA 20%	Correction OA uniquement.
C++/LINUX 10%	TSan suitability and measurement perturbation.
VISIBILITE 10%	Draft Post #3; choose one strong before/after story.
DONE	Milestone C demoable under normal + selected faults.

Bloc D - Resilience + Reporter + release

But: rendre le systeme cassable, recuperable, mesurable, reproductible et presentable.

Lundi 14 septembre	Event journal + deterministic replay
BUILD 60%	Append-only event log; replay inputs/transitions; state hash/checksum.
ALGO/OA 20%	Medium timed: array/hash or binary search.
C++/LINUX 10%	File IO separation from hot path; buffering/durability tradeoffs.
VISIBILITE 10%	PUBLIER POST #3: one measured systems experiment + methodology.
DONE	Replay produces same final state hash on reference scenario.

Mardi 15 septembre	Snapshot + crash recovery
BUILD 60%	Versioned snapshot; load + journal tail; kill -9/restart demo.
ALGO/OA 20%	30 min DP/graph error review.
C++/LINUX 10%	Crash consistency, fsync tradeoffs, schema/versioning.
VISIBILITE 10%	1 candidature; record crash-recovery demo clip.
DONE	Pre-crash vs recovered state passes automated equality checks.

Mercredi 16 septembre	Backpressure + burst behavior
BUILD 60%	Queue occupancy metrics; burst scenario; explicit overflow/stall/drop policy.
ALGO/OA 20%	Medium timed: heap/interval or queue problem.
C++/LINUX 10%	Backpressure, bounded queues, tail latency under saturation.
VISIBILITE 10%	README section: overload behavior and limitations.
DONE	Burst run reports occupancy, drops/stalls and remains internally coherent.

Bloc D - Resilience + Reporter + release - suite

Jeudi 17 septembre	Performance Reporter complete
BUILD 60%	Histograms; stage p50/p90/p99/p99.9/p99.99; throughput/network/queue/trading counters; machine metadata.
ALGO/OA 20%	30 min recall only.
C++/LINUX 10%	Histogram design, timestamp overhead, perf counter ingestion.
VISIBILITE 10%	PUBLIER mini-post #4 si resultats solides: "Reporter / tail latency".
DONE	One run creates summary.json + CSV + human-readable report.

Vendredi 18 septembre	Experiment runner + regression compare
BUILD 60%	Multi-iteration runner; baseline/current compare; regression flags; ASan/UBSan/TSan validation builds.
ALGO/OA 20%	Medium timed: mixed interview problem.
C++/LINUX 10%	perf record/report; flamegraph concepts; benchmark variance.
VISIBILITE 10%	2 candidatures cibles avec repo desormais tres presentable.
DONE	At least 3 experiments repeatable; sanitizer test suite green or documented exception.

Samedi 19 septembre	One-command demo + docs + final profiling
BUILD 60%	run_demo.sh; README; architecture; methodology; limitations; fix top profiler hotspot only if justified.
ALGO/OA 20%	Simulation OA 60 min maximum.
C++/LINUX 10%	Final perf pass: cycles, IPC, cache/branch misses, page faults.
VISIBILITE 10%	Ecrire le post final complet + CV bullet + screenshots/graphs.
DONE	Fresh clone/build instructions tested; demo produces report without manual surgery.

Bloc D - Resilience + Reporter + release - suite

Dimanche 20 septembre	FEATURE FREEZE - v1.0
BUILD 60%	No new feature. Run test matrix, fault matrix, benchmarks, tag release; archive reproducible results.
ALGO/OA 20%	Pas de nouveau LeetCode. 20-30 min: relire erreurs recurrentes seulement.
C++/LINUX 10%	Verifier compiler/version/machine metadata and benchmark caveats.
VISIBILITE 10%	Preparer publication lundi 21: GitHub link, architecture, strongest metric, failure demo, lessons.
DONE	Tag v1.0; MUST gates green; final report and demo artifacts frozen.

Publication / candidature - calendrier recommande

Date	Action	Angle
31 aout	Post #1	"Building a low-latency trading systems lab in C++" + architecture + first UDP/feed path.
7 septembre	Post #2	Gap recovery / SPSC / first measured comparison.
14 septembre	Post #3	Cache/concurrency/perf experiment with methodology, not bragging.
17 septembre	Mini-post optionnel	Performance Reporter: why p99/p99.9 and reproducible experiments matter.
20 septembre	Release v1.0	Freeze; capture clean artifacts; do not waste the day chasing a new feature.
21 ou 22 septembre	FINAL POST	Architecture + demo + strongest results + failure/recovery + GitHub + what you learned.

Strategie de candidatures pendant le sprint

- Ne pas attendre le 20. Des le premier milestone, le CV peut contenir "Low-Latency Trading Systems Lab - C++/Linux - ongoing".
- Viser quelques candidatures fortes et pertinentes chaque semaine plutot qu'un spam qui vole le temps du build.
- A chaque milestone, mettre a jour une seule ligne CV + README + pinned repo.
- Si un entretien arrive, le projet devient aussi ta banque d'histoires techniques: trade-off, bug, benchmark, failure, recovery.

Si tu prends du retard

Retard	Decision
1 jour	Absorber sur une journee d'integration; ne pas couper tests ni reporter.
2-3 jours	Supprimer SHOULD: async logger riche, second book trop ambitieux, report HTML sophistique.
4+ jours	Protéger le chemin MUST: exchange, UDP/TCP, feed, book, OMS/risk, recovery, reporter, tests, demo.
Toujours	Ne jamais ajouter NUMA/DPDK/kernel bypass avant que v1.0 end-to-end soit propre.

Checklist du 20 septembre

- Fresh clone -> configure -> build -> tests -> demo fonctionne.
- Release build, compiler/kernel/machine metadata documentes.
- Fault matrix: gap, duplicate, reorder, disconnect, burst, crash/restart.
- Reporter: p50/p99/p99.9 + throughput + queues + network + correctness.
- Au moins 3 experiences avec baseline, methode, resultats et limites.
- README lisible en 5 minutes: what/architecture/build/demo/results/design decisions.
- Tag v1.0 + release notes + screenshots/graphs propres.
- Post final pret pour le lundi 21 ou mardi 22.